

COM S 327, Spring 2026

Programming Project 1.05

User Interface

Last week we added some characters, and made them move around. You may have added some code to drive your @; You can rip that code out, now¹. We're going to add a user interface that you can use to drive your @ manually. If you like, you can leave the auto-drive code in there and add a command to turn it on and off at runtime.

Still working in C, link in the ncurses library and use it for unbuffered I/O.

Neither the PC nor the NPCs may use the map gates at this point. Do not allow the PC to stand on the gate; this will do *Bad Things* to your pathfinding implementation from 1.03.

As for user input, all commands are to be activated immediately upon key-press. There is never a need to hit enter (unbuffered input). Any command which is not explicitly defined is a no-op. Implement the following commands:

Key(s)	Action
7 or y	Attempt to move PC one cell to the upper left.
8 or k	Attempt to move PC one cell up.
9 or u	Attempt to move PC one cell to the upper right.
6 or l	Attempt to move PC one cell to the right.
3 or n	Attempt to move PC one cell to the lower right.
2 or j	Attempt to move PC one cell down.
1 or b	Attempt to move PC one cell to the lower left.
4 or h	Attempt to move PC one cell to the left.
>	Attempt to enter a Pokémart or Pokémon Center. Works only if standing on a building. Leads to a user interface for the appropriate building. You may simply add a placeholder for this for now, which you exit with a <.
5 or space or .	Rest for a turn. NPCs still move.
t	Display a list of trainers on the map, with their symbol and position relative to the PC (e.g.: "r, 2 north and 14 west").
up arrow	When displaying trainer list, if entire list does not fit in screen and not currently at top of list, scroll list up.
down arrow	When displaying trainer list, if entire list does not fit in screen and not currently at bottom of list, scroll list down.
escape	When displaying trainer list, return to character control.
Q	Quit the game. Your main game loop will become something like: <pre>while (!quit_game) { ... }</pre>

The following set of keys are reserved for current and future required functionality:

- `_`²
- `.`
- `1`
- `2`

¹If you gave your PC any special powers, like the ability to move through mountains or forests, those should no longer apply, either; your PC is a normal human, save that he or she may be abnormally foolish.

²This is a space, rendered in a visible manner. A sort of *visible space*, if you will.

- 3
- 4
- 5
- 6
- 7
- 8
- 9
- >
- Q
- b
- f
- g
- h
- j
- k
- l
- n
- p
- t
- u
- y

You may use any other keys not listed above, either as aliases for required functionality, or to implement features or debugging routines that are unique to your game.

If the player attempts to move to a map cell with an NPC, or if an NPC attempts to move to the PC's map cell, and the PC has not already defeated this trainer in a battle, a *Pokémon Battle* interface is triggered. For now, this interface is a placeholder. The only command in the interface is `escape`, to leave the battle, which marks the NPC as having been defeated. A defeated hiker or rival will no longer path to the PC (it's up to you how you would like them to move from this time on). If the PC attempts to move into the cell of a defeated trainer, nothing happens.

With these changes, we no longer need the delay that we built in last week; the game will now pause automatically for input. And ncurses should handle the redrawing, so we're no longer spewing the entire map to the terminal each turn. Things will look much nicer.

Note that the keys `y`, `k`, `u`, `l`, `n`, `j`, `b`, and `h` are not as strange or arbitrary as they may initially appear. `vi` and `vim` users will immediately recognize these as the cursor movement keys in their editor. They're also used in many roguelike games (including the original *Rogue* and most of its direct descendants) and have been since long before most of you were born.

Our maps fill 21 out of 24 lines in a terminal. Display them on lines 1–21 (zero indexed). The top line is for message display. Use it to display any messages you like (like debugging information, or information about why a command can't be executed ("There's a tree in the way!" or "A wild Riakou appears!")). The bottom 2 lines are for status information, which we'll deal with in a later assignment.

You may add any other commands you like, or map the required commands to additional keys, as long as you implement the specified mappings; however, you may be forced to change keys for any extra commands in future assignments.

Color!

Curses makes it easy to implement color. It is not required, but I highly recommend you color your

terrain now. We'll talk about how to do this (and other Curses stuff) in lecture. If you already have color using Curses, the rest of this should be really easy for you. If you have color implemented some other way, you're going to have to change it to use Curses color.